

Audit de ProspectWeb

Revue technique, sécurité, produit & business
fondée sur la lecture intégrale du code source

OUTIL AUDITÉ

ProspectWeb
github.com/sofianeHms/prospectweb

DATE

22 juin 2026

MÉTHODE

Lecture du dépôt
frontend + backend, commit cloné

DIFFUSION

Restreinte

Sommaire

1.	Périmètre, méthode et angles morts	3
2.	Résumé exécutif	4
3.	Audit technique & architecture	5
4.	Audit de sécurité	7
5.	Audit produit, UX & expérience client	10
6.	Audit business & stratégique	12
7.	Conclusion croisée & recommandation globale	14

Grille de criticité utilisée dans tout le document : **Bloquant** empêche une mise en production ou commercialisation sérieuse — **Majeur** impact fort à corriger avant toute montée en charge — **Mineur** à traiter mais non urgent — **Cosmétique** finition.

Chaque constat distingue le **fait** (vérifié dans le code, référencé [fichier:ligne](#)) de l'**opinion / recommandation**.

1 Périmètre, méthode et angles morts

Cet audit repose exclusivement sur la lecture du code source du dépôt [sofianehtms/prospectweb](#) (frontend Next.js et backend Express/TypeScript), cloné et lu fichier par fichier : les douze fichiers `.ts/.tsx` du backend et l'intégralité de `frontend/app`, plus les fichiers de configuration (`tsconfig`, `package.json`, `.gitignore`, `.env.example`, CI GitHub Actions) et le `README` (≈ 650 lignes). Un `npm audit` a été exécuté sur les deux `package-lock`.

Stack technique réellement constatée

COUCHE	CONSTAT DANS LE CODE
Frontend	Next.js ^15.3.3 (App Router), React 19.2.4, Tailwind v4, Leaflet 1.9.4, TypeScript 5. Déployé sur Vercel. (Le README annonce « Next.js 16.2.6 » — voir §3.)
Backend	Express ^5.2.1, TypeScript (<code>strict: true</code> , cible ES2020), <code>cors</code> 2.8.6, <code>express-rate-limit</code> 8.5.2, <code>dotenv</code> 17. Dev via <code>ts-node</code> + <code>nodemon</code> . Déployé sur Railway.
Persistence	Aucune base de données. Cache serveur en mémoire (<code>services/cache.ts</code> , <code>Map</code> , TTL 10 min). Données utilisateur (CRM) en <code>localStorage</code> navigateur (<code>hooks/useProspects.ts</code>).
APIs tierces	Google Places API (New) <code>searchNearby</code> (<code>services/places.ts</code>), Google Geocoding API (<code>services/geocode.ts</code>), Nominatim/OSM (autocomplétion appelée <i>depuis le navigateur</i> , <code>SearchForm.tsx:70</code>) et tuiles OSM (<code>Map.tsx</code>).
Authentification	Un unique secret partagé front ↔ back (<code>x-internal-secret</code> , <code>middleware/auth.ts</code>). Aucun compte utilisateur.

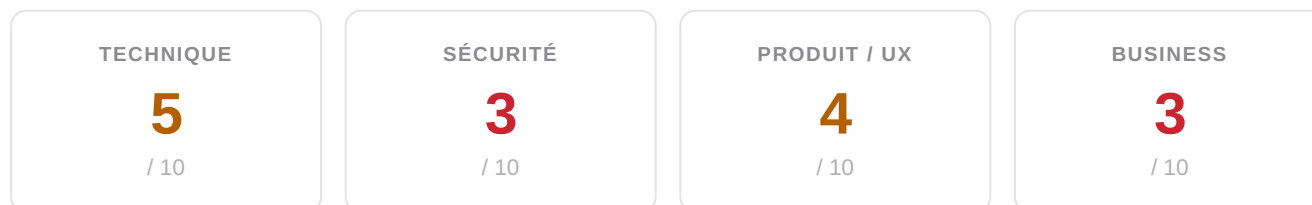
ANGLES MORTS ASSUMÉS DE CET AUDIT

Les éléments suivants ne sont pas vérifiables depuis le code et ne sont donc **pas** jugés — ils sont signalés comme limites :

- Configuration réelle d'infrastructure (nombre d'instances Railway, autoscaling, mémoire allouée, variables d'environnement effectivement renseignées en production).
- Plafonds de quota / QPD réellement configurés dans la console Google Cloud (élément *déterminant* pour le risque de coût — voir §4 et §6).
- Volumétrie d'usage réelle, logs de production, taux d'erreur observés, facture Google constatée.
- Comportement de l'application en exécution réelle : l'audit n'a pas lancé l'app en production ; les conclusions UX découlent de la lecture du rendu et de la logique, pas d'un test live.
- Tarifs exacts des SKU Google : les ordres de grandeur de \$6 sont à recouper avec la page de pricing Google en vigueur.

2 Résumé exécutif

ProspectWeb est un **MVP mono-utilisateur compétent et soigné** : code propre, TypeScript strict, interface élégante, parcours de base fonctionnel, export CSV. Mais en l'état il n'est **ni prêt pour une mise en production sérieuse ni commercialisable en SaaS**. Trois familles de problèmes dominent : un **risque financier non borné** (application ouverte sans compte, amplifiant les appels Google payants), des **données fabriquées affichées à l'utilisateur**, et des **expositions juridiques** (conditions Google Places, RGPD, mentions de sources fausses).



Les 5 problèmes les plus urgents

#	PROBLÈME	CRITICITÉ	POURQUOI C'EST URGENT
1	Application ouverte = robinet de coût Google ouvert	Bloquant	Aucun compte/login. N'importe qui sur Internet lance des recherches payées par votre clé Google. Le paramètre <code>types</code> n'est pas filtré : 50 types fournis = 250 appels Google par requête (<code>search.ts:62</code> , <code>places.ts:173-180</code>). Le rate-limiter est en mémoire et mal placé (\$4.2). Risque : facture de plusieurs centaines/milliers d'€ en une nuit.
2	Données inventées dans la fiche prospect	Bloquant	Horaires « Lun–Sam 7h–20h » (<code>DetailClient.tsx:111</code>) et « Instagram présent (783 abonnés) » (<code>:126</code>) sont codés en dur, identiques pour tous . Un utilisateur qui prospecte sur de fausses infos perd toute confiance dans l'outil.
3	Exposition juridique (Google ToS / RGPD / fausses sources)	Bloquant	Stockage/export durable de données Places (nom, téléphone, adresse) au-delà du Place ID, prospection RGPD sans base légale ni opt-out, et pied de page affichant « Pages Jaunes · SIRENE » (<code>page.tsx:38</code>) alors qu'aucune de ces sources n'est utilisée.
4	Le signal central (« sans site / obsolète ») est peu fiable	Majeur	« Récent » = simple présence de « 2025 »/« 2026 » dans les 50 premiers Ko (<code>websiteChecker.ts:103</code>) ; site injoignable → classé « obsolète » (<code>search.ts:25</code>) ; erreurs Google avalées silencieusement → « 0 résultat » (<code>places.ts:115,118</code>). La promesse de qualification repose sur une heuristique fragile.
5	Fonctions affichées mais non câblées	Majeur	« Filtres rapides » de l'accueil jamais envoyés au backend (<code>SearchForm.tsx:108-116</code>) ; statut CRM & notes de la fiche détail non persistés (état local, <code>DetailClient.tsx:56-57</code>) ; bouton « Envoyer un email » sans action ; liens profonds <code>/results/[id]</code> cassés hors session.

Verdict synthétique : **refonte partielle requise** côté backend (comptes, persistance, contrôle des coûts), correction du contenu fabriqué et mise en conformité juridique avant d'envisager une commercialisation. Détail et recommandations actionnables dans les sections suivantes.

3 Audit technique & architecture

3.1 — Qualité générale du code

Constats

Le code est, pour sa taille, **de bonne facture** : `strict: true` est bien activé (`backend/tsconfig.json`), les interfaces sont partagées et explicites (`types/establishment.ts`, `places.ts:46`), le typage `unknown` est utilisé là où il faut (`cache.ts:7`, `search.ts:72`) plutôt que `any` — je n'ai relevé **aucun** `any` ni `@ts-ignore`. Les fonctions sont courtes et la complexité cyclomatique reste basse ; la plus ramifiée, `detectRecentContent` (`websiteChecker.ts:84`), reste lisible. Aucun `TODO/FIXME/HACK` dans le code.

La séparation des responsabilités backend est correcte : `routes/`, `services/`, `middleware/`. Il n'y a pas de couche repository, mais l'absence de base de données la rend ici non nécessaire. Le contrôleur `search.ts` concentre validation + orchestration + calcul du résumé : acceptable à cette échelle, mais c'est le point où la logique commence à s'accumuler.

3.2 — Dette technique, code mort & incohérences

CONSTAT (FAIT)	CRITICITÉ	DÉTAIL & RÉFÉRENCE
Aucun test, nulle part	Majeur	Zéro fichier <code>.test/</code> . <code>spec</code> . La CI (<code>.github/workflows/ci.yml</code>) n'exécute que <code>lint</code> , <code>tsc --noEmit</code> et <code>build</code> . Aucune couverture sur la logique critique (heuristique de récence, multi-zones, cache). Inacceptable pour un outil destiné à être vendu.
Code mort : <code>lib/mock.ts</code>	Mineur	<code>MOCK_ESTABLISHMENTS</code> n'est importé nulle part (vérifié par grep). 12 fiches fictives résiduelles.
Code mort & divergent : <code>ThemeProvider.tsx</code>	Mineur	Composant non importé. Pire : il définit un autre schéma de thèmes (<code>'light' 'beige' 'dark' 'darker'</code>) et lit <code>localStorage</code> , alors que le thème réel (<code>layout.tsx</code> , <code>ThemeToggle.tsx</code>) utilise <code>'sable' 'violet' 'minuit' 'cosmos'</code> via un cookie. Source de confusion de maintenance.
<code>backend/.env.example</code> incomplet	Majeur	Ne contient que <code>PORT</code> et <code>FRONTEND_URL</code> . Il manque <code>GOOGLE_PLACES_KEY</code> et <code>BACKEND_SECRET</code> , pourtant requis par le code (<code>places.ts:172</code> , <code>auth.ts:4</code>). Un dev qui suit ce fichier obtient un backend cassé (503) et non fonctionnel.
README : table des variables fausse	Majeur	Le README liste <code>BACKEND_SECRET</code> à configurer <i>sur Vercel uniquement</i> , pas sur Railway. Or <code>auth.ts:4-9</code> lit <code>BACKEND_SECRET</code> côté <i>backend</i> et renvoie 503 sur chaque requête s'il est absent. Suivre la doc à la lettre rend la prod non fonctionnelle.
<code>.env.example</code> racine : variable fantôme	Mineur	Déclare <code>OVERPASS_API_URL</code> (API Overpass) qui n'est utilisée nulle part dans le code. Vestige trompeur.
Désynchronisation README ↔ code	Cosmétique	README : « Next.js 16.2.6 » ; <code>package.json</code> : <code>next ^15.3.3</code> (c'est <code>eslint-config-next</code> qui est en 16). Le README documente le géocodage comme Google (correct) mais la description marketing parlait de « Nominatim/OSM » pour le géocodage — en réalité Nominatim ne sert qu'à l'autocomplétion.

3.3 — Gestion des erreurs

Constats

Le contrôleur `search.ts` encadre bien ses appels (try/catch, codes 400/502). `websiteChecker.ts` gère proprement timeout (`AbortController` , 6 s) et exceptions. **Mais** le service Places masque ses erreurs :

ÉCHECS GOOGLE AVALÉS SILENCIEUSEMENT

`places.ts:115` : `if (!res.ok) return [];` et `:118` : `catch { return []; }`. Un `403` (quota dépassé, facturation suspendue, clé invalide/restrictée) ou un `429` Google est transformé en **liste vide**, indiscernable d'« aucun commerce dans la zone ». L'utilisateur — et l'exploitant — ne voient jamais la panne. **Majeur**

Recommandations — Technique

- 1 Remonter les erreurs Google : journaliser `res.status` et le corps dans `places.ts`, et distinguer « 0 résultat » d'« échec fournisseur » jusqu'au frontend (bannière dédiée).
- 2 Corriger `backend/.env.example` (ajouter `GOOGLE_PLACES_KEY` , `BACKEND_SECRET`) et la table d'environnement du README (`BACKEND_SECRET` + `GOOGLE_PLACES_KEY` côté Railway).
- 3 Introduire des tests : unitaires sur `detectRecentContent` , `resolveWebsiteStatus` , `subCenters` et le cache ; un test d'intégration sur `/api/search` avec Google mocké. Brancher la couverture dans la CI.
- 4 Supprimer le code mort (`lib/mock.ts`, `ThemeProvider.tsx`) et la variable `OVERPASS_API_URL` .
- 5 Aligner les versions annoncées (README) sur `package.json` pour éviter les fausses promesses de stack.

4 Audit de sécurité

4.1 — Authentification & autorisation

Constats

Point positif : le secret `BACKEND_SECRET` n'est pas exposé au navigateur. La page résultats est un Server Component qui appelle le backend côté serveur (`results/page.tsx:16-19`) avec `process.env.BACKEND_SECRET` ; aucune variable n'est préfixée `NEXT_PUBLIC_`. Le middleware échoue « fermé » : si `BACKEND_SECRET` manque, tout est rejeté en 503 (`auth.ts:6-9`). Aucune clé en dur dans le code, `.gitignore` exclut bien les `.env`.

LE DÉFAUT STRUCTUREL : AUCUNE AUTHENTIFICATION UTILISATEUR

Le secret protège le backend des *appels directs de tiers*, mais **pas** le frontend public de l'abus. N'importe qui atteignant `prospectweb.vercel.app` peut lancer des recherches illimitées, payées par la clé Google de l'exploitant. Il n'existe ni compte, ni login, ni quota par utilisateur, ni attribution d'usage. L'application est, de fait, un **proxy ouvert et anonyme vers Google Places, générateur de coûts**. **Bloquant** pour toute commercialisation.

Comparaison de constant-temps : `auth.ts:13` compare le secret avec `!==` (non constant-temps). Attaque par timing théorique sur un secret long et aléatoire — **Mineur**.

4.2 — Rate limiting : cosmétique en l'état

FAIT (CODE)	CONSÉQUENCE
Store en mémoire par défaut (<code>rateLimit.ts</code> , aucun store Redis)	Limite par instance, remise à zéro à chaque redéploiement/restart Railway ; non partagée si >1 instance.
Mauvaise clé d'identification. Le limiteur est sur le backend, que <i>seul le frontend</i> appelle (avec le secret). Les utilisateurs finaux passent tous par les IP serveur de Vercel.	Le quota de 20/min se calcule sur les IP de Vercel partagées par tous les utilisateurs → soit blocage injuste d'utilisateurs légitimes, soit inefficacité totale contre un attaquant. Majeur
<code>trust proxy</code> jamais configuré (<code>index.ts</code>)	Derrière le proxy Railway, <code>req.ip</code> est probablement l'IP du proxy → tout le trafic dans un seul seau.
20 req/min sans CAPTCHA ni preuve de travail	Contournable par simple rotation d'IP.

4.3 — Validation des entrées & amplification de coût

Constats

La validation de base est présente : `radius` borné 1–10 000 (`search.ts:36`), `lat/lng` contrôlés par `isNaN` (:45), `address` encodé avant géocodage. Mais :

LE PARAMÈTRE `TYPES` N'EST PAS FILTRÉ CONTRE UNE LISTE BLANCHE

`search.ts:62-64` découpe `types` tel quel ; `places.ts:173-180` envoie **chaque** type fourni dans `fetchByTypeMultiArea` = jusqu'à 5 appels Google par type. Fournir `types=t1,t2,...,t50` (types arbitraires, même invalides — Google facture quand même) déclenche **~250 appels Google payants en une seule requête**. Couplé à l'absence d'authentification (§4.1), c'est un vecteur de **déni de service financier**. **Bloquant**

4.4 — SSRF, en-têtes, fuites

SUJET	CRITICITÉ	CONSTAT
SSRF via site contrôlé	Majeur	<code>websiteChecker.ts:22-26</code> fait un <code>fetch</code> avec <code>redirect: 'follow'</code> vers l'URL <code>websiteUri</code> d'un commerce. Un attaquant créant une fiche Google avec un site redirigeant vers une IP interne (ex. métadonnées cloud, <code>169.254.x.x</code> , réseau Railway) fait sonder le réseau interne par le backend. Lecture limitée (50 Ko, booléens/dates), mais aucune liste blanche ni blocage des IP privées.
En-têtes de sécurité absents	Mineur	Pas de <code>helmet</code> côté Express ; <code>next.config.ts</code> est vide (aucun <code>headers()</code>). Ni CSP, ni HSTS, ni X-Frame-Options définis par l'app (au-delà des défauts Vercel).
Messages d'erreur verbeux	Mineur	<code>search.ts:53,105</code> renvoient <code>(err as Error).message</code> au client ; <code>geocode.ts:20</code> y inclut le <code>status</code> et l' <code>error_message</code> Google. Fuite d'informations internes possible (état de la clé, motifs Google).
CORS	Cosmétique	Origine unique <code>FRONTEND_URL</code> (<code>index.ts:13</code>). Peu pertinent ici puisque les appels backend sont serveur-à-serveur ; défense en profondeur seulement.
Nominatim appelé depuis le navigateur	Majeur	<code>SearchForm.tsx:70</code> : appel direct à <code>nominatim.openstreetmap.org</code> sans User-Agent identifiant/e-mail, exposant l'IP des utilisateurs et violant la politique d'usage OSM (voir aussi §6).

4.5 — Deux scénarios d'attaque concrets

SCÉNARIO A — ÉPUISEMENT FINANCIER (RÉUSSIT AUJOURD'HUI)

L'app Vercel est publique et sans compte. Un attaquant scripte des requêtes vers `/results?lat=...&lng=...&radius=10000&types=t1,...,t50`. Chaque requête → serveur Next → backend → 50 types × 5 sous-centres = **250 appels Google Places (New)**, dans le palier tarifaire élevé car le `FieldMask` demande `websiteUri` / téléphone / `rating` (`places.ts:5-16`). Le limiteur backend, calé sur les IP de Vercel, ne distingue pas l'attaquant et peut au contraire bloquer les vrais clients. Aucun plafond QPD appliqué *dans le code*. Quelques milliers de requêtes nocturnes = **facture Google de plusieurs centaines à milliers d'€**. **Bloquant**

SCÉNARIO B — SONDE SSRF VIA FICHE GOOGLE CONTRÔLÉE (RÉUSSIT PARTIELLEMENT)

L'attaquant crée/possède une fiche Google dont le site pointe vers une URL interne ou redirigeante. Dès qu'une recherche d'un utilisateur inclut ce commerce, le backend récupère l'URL côté serveur (`redirect: follow`), permettant un balayage réseau interne en aveugle et de l'inférence par timing depuis le réseau Railway. Exfiltration limitée (booléen « récent » + temps de réponse), mais le vecteur SSRF est réel et non atténué. **Majeur**

Recommandations — Sécurité (par ordre de priorité)

- 1 Introduire des comptes utilisateurs** et placer toute recherche derrière une authentification, avec quota par utilisateur. C'est le préalable à toute commercialisation et la première barrière anti-coût.
- 2 Liste blanche stricte de `types`** : rejeter (400) tout type hors `COMMERCIAL_TYPES` et plafonner le nombre de types par requête (`search.ts`).
- 3 Plafond de coût défensif** : QPD/quotas dans Google Cloud Console + budget alert ; idéalement un compteur d'appels applicatif par utilisateur/jour.

- 4 **Rate-limiting durable et correct** : store partagé (Redis), clé sur l'identité utilisateur (pas l'IP Vercel), `app.set('trust proxy', 1)`.
- 5 **Durcir `websiteChecker`** : interdire les IP privées/loopback/link-local après résolution DNS, limiter les redirections, idéalement passer par un proxy de sortie cloisonné.
- 6 Ajouter `helmet` (Express) et un bloc `headers()` Next (CSP, HSTS, X-Frame-Options) ; renvoyer des messages d'erreur génériques au client et journaliser le détail côté serveur.
- 7 Proxifier Nominatim côté backend avec un User-Agent identifiant, ou self-héberger (§6).

5 Audit produit, UX & expérience client

Posture client final Je paie pour trouver des prospects exploitables, vite et avec confiance. L'interface est belle et claire — mais plusieurs promesses ne tiennent pas une fois que je gratte.

5.1 — Fiabilité perçue : le point qui casse la confiance

CONSTAT (FAIT)	CRITICITÉ	IMPACT CLIENT
Données fabriquées dans la fiche	Bloquant	« Lun–Sam 7h–20h » (DetailClient.tsx:111), « Instagram présent (783 abonnés) » (:126), « Fiche Google Maps complète » (:127) sont identiques pour <i>tous</i> les établissements. Je découvre tôt ou tard que c'est faux → je ne fais plus confiance à aucune donnée de l'outil.
Heuristique « récent » grossière	Majeur	Un simple « © 2026 » en pied de page d'un site par ailleurs vétuste le classe « actif » ; à l'inverse un bon site sans année visible est classé « obsolète » (websiteChecker.ts:103-105). Beaucoup de faux positifs/négatifs sur le cœur de la proposition de valeur.
Injoignable = « obsolète »	Majeur	search.ts:25-26 : un site momentanément down ou qui bloque le bot est étiqueté « Site obsolète ». Je peux appeler un commerce en lui disant que son site est vieux... alors qu'il est très bien.
Résultats partiels non signalés	Majeur	Sans type sélectionné, le backend fait <i>un</i> appel par type depuis le centre unique (places.ts:180 , branche useMulti=false) → plafonné à ~20/type, sans densification multi-zones. En zone dense je rate des commerces, sans aucun avertissement « résultats incomplets ».

5.2 — Fonctions affichées mais non câblées

CONSTAT (FAIT)	CRITICITÉ	DÉTAIL
« Filtres rapides » inopérants	Majeur	5 filtres affichés (Sans site, Site obsolète, Pas de Google Maps, Pas de réseaux sociaux, Ouvert récemment, SearchForm.tsx:27-33) mais activeFilters n'est jamais ajouté aux paramètres dans handleSubmit (:108-116). Ils ne font rien ; le backend n'a aucune notion de « réseaux sociaux » ou « récemment ouvert ».
CRM & notes de la fiche non persistés	Majeur	DetailClient.tsx:56-57 : statut et notes en useState local, déconnectés de useProspects/localStorage . Tout est perdu à la navigation. (La page liste « Mes prospects » persiste, elle ; l'incohérence entre les deux écrans est déroutante.)
Bouton « Envoyer un email » mort	Mineur	DetailClient.tsx:209-214 : aucun href/onClick , et aucune donnée e-mail n'existe dans le modèle.
Liens profonds cassés	Majeur	La fiche /results/[id]/page.tsx lit sessionStorage('pw_search_results') . Ouvrir un résultat dans un nouvel onglet, ou après expiration de session, affiche « Commerce introuvable ». Impossible de partager le lien d'un prospect.

5.3 — Parcours, performance perçue, cohérence

Constats

Absence d'état de chargement réel. La recherche peut durer plusieurs secondes (5×N appels Google + vérifications de sites à timeout 6 s, tous en `Promise.all`). Or la page résultats est un Server Component qui *bloque* jusqu'à obtention des données : après le clic, l'utilisateur reste sur une page figée sans indicateur de progression (le spinner du bouton d'accueil disparaît à la navigation). **Majeur** sur la performance perçue.

Incohérences de libellés de types. Les tables de libellés/icônes (`ResultsClient.tsx:28-37`, `DetailClient.tsx`) référencent des types absents de `COMMERCIAL_TYPES` (ex. `laundry`, `locksmith`, `books`, `bank`, `veterinary`) tandis que des types réellement interrogés (`hotel`, `clothing_store`, `jewelry_store`, `plumber`, `electrician`...) n'ont parfois ni libellé ni icône → affichage de chaînes brutes (`car_repair` → « car repair ») ou icône générique.

Mineur.

Accessibilité : les sliders ont des `aria-label` (bien), mais beaucoup d'icônes SVG décoratives ne sont pas masquées aux lecteurs d'écran, et la carte encode le statut par la seule couleur. **Mineur**.

5.4 — Comparaison aux standards du marché & manques

Posture client Les bons points existent : carte propre, tri multi-critères, filtres de statut côté résultats, **export CSV** (`useProspects.ts`), génération de script d'appel. Mais par rapport à ce qu'un professionnel attend d'un outil de prospection B2B, il manque :

- **Comptes & persistance serveur** : tout le CRM vit en `localStorage` → perdu en changeant de navigateur/appareil, non partageable en équipe, effacé par un simple nettoyage de cache.
- **Déduplication inter-recherches et historique de recherche** : aucun. Je reverrai les mêmes prospects à chaque recherche voisine.
- **Enrichissement** (e-mail, dirigeant, SIRET) : absent ; les actions « email » sont des leurres.
- **Suivi de campagne / relances datées** : le CRM se limite à 5 statuts sans échéances ni rappels.

Recommandations — Produit / UX (par priorité)

- 1 **Supprimer immédiatement toute donnée fabriquée** (horaires, Instagram, « fiche complète ») : n'afficher que ce qui provient réellement de l'API, sinon « Non renseigné ».
- 2 **Câbler ou retirer les fonctions fantômes** : faire agir les filtres rapides (ou les masquer), persister le CRM/notes de la fiche via `useProspects`, activer ou supprimer le bouton e-mail.
- 3 **Fiabiliser le signal** : séparer « injoignable » d'« obsolète », remplacer la détection « année dans le texte » par un faisceau d'indices pondérés (date `Last-Modified`, sitemap, technologies), et exposer un niveau de confiance.
- 4 **Rendre les liens profonds robustes** : recharger la fiche depuis le backend par `id` si la session est absente.
- 5 **Vrai état de chargement** : streaming/skeleton sur la page résultats et indicateur de progression pendant la recherche.
- 6 Signaler explicitement quand les résultats sont plafonnés/partiels ; harmoniser les tables de libellés avec `COMMERCIAL_TYPES`.

6 Audit business & stratégique

Posture patron / investisseur Trois questions me suffisent : combien ça coûte quand ça scale, qu'est-ce qui me protège d'un concurrent, et puis-je le vendre sans me mettre en infraction ? Les trois réponses sont aujourd'hui défavorables.

6.1 — Modèle de coûts : le risque existentiel

Constats & analyse

Le `FieldMask` demandé (`places.ts:5-16`) inclut `websiteUri`, `nationalPhoneNumber`, `rating`, `userRatingCount` : ces champs « contact » et « atmosphère » font basculer chaque `Nearby Search` dans un **palier tarifaire élevé**, pas dans le tier « IDs only » bon marché. Multiplicateurs structurels du code :

MÉCANIQUE (CODE)	EFFET SUR LE COÛT
5 sous-centres par type sélectionné (<code>places.ts:126-137,147-162</code>)	×5 appels par type demandé.
<code>types</code> non plafonné (§4.3)	Multiplication non bornée par requête (50 types ⇒ 250 appels).
Re-vérification de tous les sites à chaque <code>cache-miss</code>	Charge CPU/sockets backend, pas de coût Google direct mais limite la concurrence.
Cache en mémoire , TTL 10 min (<code>cache.ts</code>)	Perdu à chaque redéploiement ; non partagé entre instances. La principale mitigation de coût est la plus fragile possible.

Contexte tarifaire vérifié : Google a **supprimé le crédit universel de 200 \$/mois** (mars 2025) au profit de plafonds gratuits par SKU bien plus petits (typiquement quelques milliers d'événements/mois pour les SKU concernés). Combiné à l'application ouverte (§4.1), **le code ne borne en rien la dépense** ; à 10× et a fortiori 100× du volume, c'est le risque numéro un. (Les tarifs SKU exacts sont à recouper avec la page de pricing Google en vigueur — angle mort §1.) **Bloquant**

6.2 — Dépendance fournisseurs & continuité

Dépendance dure à Google (Places + Geocoding), Nominatim, tuiles OSM, Vercel et Railway, **sans plan de continuité** : une seule clé Google ; si elle est bloquée, en quota ou facturation suspendue, `places.ts` renvoie des listes vides → le produit « marche » mais ne retourne plus rien, sans alerte (§3.3). Aucun fournisseur de repli (géocodeur/places alternatif), aucun circuit-breaker. **Majeur**.

6.3 — Scalabilité de l'architecture actuelle

Le backend est sans état *sauf* le cache et le rate-limiter, tous deux en mémoire → **la mise à l'échelle horizontale casse à la fois le cache et le rate-limiting**. La vérification des sites lance N `fetch` parallèles par recherche (timeout 6 s) sans plafond de concurrence ni file d'attente : quelques recherches « tous types » simultanées (23 types × nombreux établissements × fetch HTTP) peuvent saturer l'event-loop et la mémoire d'une instance Railway. Pas de base de données, pas de file de jobs. **Majeur**.

6.4 — Positionnement & défensabilité

Posture patron La proposition (Google Places → filtrer les « sans site » → mini-CRM → export) est **reproductible en quelques jours**. La seule « IP » est l'heuristique de récence, justement la plus fragile (§5.1). Pas de donnée propriétaire, pas d'effet de réseau, pas d'intégration verrouillante. **Faible barrière à l'entrée**. **Majeur** (stratégique).

6.5 — Risques juridiques

SUJET	CRITICITÉ	CONSTAT
Conditions Google Places	Bloquant	L'export CSV et le stockage durable de noms, téléphones et adresses (useProspects.ts) vont au-delà du cache limité autorisé ; en général seul le Place ID peut être conservé durablement, et la réutilisation des données hors contexte d'une carte Google est restreinte. Le cœur d'usage (exporter pour démarcher) est précisément ce que les conditions encadrent.
RGPD / prospection	Majeur	Téléphones d'indépendants/professionnels = données personnelles potentielles. L'outil ne propose ni base légale, ni opt-out, ni durée de conservation (données indéfiniment en <code>localStorage</code>), ni mention CNIL/ politique de confidentialité.
Mention de sources fausse	Majeur	page.tsx:38 affiche « Données issues de Google Maps · Pages Jaunes · SIRENE ». Le code n'utilise ni Pages Jaunes ni SIRENE . Affirmation de provenance trompeuse, avec en plus des implications de licence propres à ces sources.
Politique d'usage Nominatim/OSM	Majeur	Appel direct depuis le navigateur sans User-Agent identifiant (SearchForm.tsx:70) ; l'API publique vise un usage raisonnable et faible volume (≤ 1 req/s, UA identifiant). Un usage commercial devrait être self-hébergé. Risque de bannissement IP.

6.6 — Viabilité en SaaS multi-clients

En l'état : pas de comptes, pas d'auth, pas de facturation, pas de quota par client, pas de persistance serveur.

L'outil ne peut pas être vendu en SaaS multi-tenant sans une reconstruction substantielle du backend

(comptes, base de données, métréage d'usage, répercussion des coûts Google). Aujourd'hui, c'est un outil interne mono-utilisateur — ce qu'il fait correctement.

Recommandations — Business (par priorité)

- 1 Maîtriser le coût avant tout** : réduire le [FieldMask](#) au strict nécessaire de chaque vue, plafonner sous-centres et types, cache durable et partagé (Redis), métréage par utilisateur, plafonds GCP + alertes budget.
- 2 Sécuriser juridiquement** : revoir le stockage/export à l'aune des conditions Google Places (ne persister que ce qui est permis), ajouter politique de confidentialité, base légale RGPD et opt-out, et **retirer la mention « Pages Jaunes · SIRENE »** (ou l'implémenter réellement et sous licence).
- 3 Plan de continuité fournisseurs** : alerting sur erreurs Google, abstraction d'un géocodeur/places de repli, self-hébergement (ou proxy identifié) de Nominatim.
- 4 Re-architecturer pour scaler** : base de données, cache/rate-limit externalisés, file d'attente et plafond de concurrence pour la vérification de sites.
- 5 Construire un différenciateur** : enrichissement propriétaire, déduplication/historique, intégrations CRM — sinon le produit reste trivialement copiable.
- 6 Pré-requis SaaS** : comptes, facturation, quotas, persistance serveur multi-tenant.

7 Conclusion croisée & recommandation globale

7.1 — Les trois postures, confrontées

POSTURE	VERDICT
Patron / investisseur	« Joli MVP, mais je ne peux pas financer un produit dont la structure de coûts est ouverte à l'abus et dont la défensabilité est nulle. Avant toute levée ou vente, je veux : des comptes, un contrôle de coût démontré, et une conformité Google/RGPD qui ne m'expose pas. En l'état, l'actif est l'idée validée et un front soigné — pas une entreprise. »
Client final	« L'interface me plaît et l'export CSV est utile. Mais le jour où je réalise que les horaires et l'Instagram sont inventés , et qu'un site classé "obsolète" était en fait très bien, je doute de tout le reste. Les filtres ne filtrent pas, mes notes disparaissent. Je ne paierai pas pour ça tant que les données ne sont pas fiables. »
Auditeur technique & sécurité	« Le code est propre et bien typé, ce qui est rare et appréciable. Mais l'absence de compte transforme l'app en proxy payant ouvert , le paramètre types amplifie le coût, le rate-limiter est cosmétique, un vecteur SSRF traîne, et il n'y a aucun test. Ce ne sont pas des détails de finition : ce sont des défauts de conception. »

7.2 — Convergence

Les trois regards convergent vers le même diagnostic : **ProspectWeb est un excellent prototype et une démonstration de savoir-faire front-end, mais pas un produit prêt**. Ce qui le bloque n'est pas la qualité d'écriture du code (réelle), mais quatre décisions de conception : (1) pas d'identité utilisateur, (2) coût non borné, (3) contenu fabriqué présenté comme réel, (4) usage des données tierces non conforme.

7.3 — Recommandation globale

DÉCISION

Non prêt pour une mise en production sérieuse ni pour une commercialisation. Adapté en revanche comme outil interne mono-utilisateur ou pièce de portfolio. Le passage à un produit vendable exige une **refonte partielle ciblée du backend** — pas une réécriture totale, mais davantage qu'une passe de finition.

Séquencement conseillé

PHASE	OBJECTIF	CONTENU
Phase 0 jours	Stopper l'hémorragie & rétablir la confiance	Retirer les données fabriquées ; liste blanche + plafond sur types ; plafonds/alertes GCP ; retirer la mention « Pages Jaunes · SIRENE » ; messages d'erreur génériques.
Phase 1 semaines	Rendre le produit commercialisable	Comptes + auth + quotas par utilisateur ; persistance serveur ; cache & rate-limit externalisés ; fiabiliser l'heuristique de récence ; câbler/retirer les fonctions fantômes ; vrais états de chargement ; conformité RGPD & Google Places.
Phase 2 trimestre	Défendre & différencier	Enrichissement propriétaire, déduplication/historique, intégrations CRM, plan de continuité fournisseurs, tests & couverture en CI.

Fin du rapport. Les constats ci-dessus reposent sur la lecture effective du code aux références indiquées ; les jugements de criticité et les recommandations relèvent de l'analyse de l'auditeur. Les éléments d'infrastructure, de volumétrie et de tarification non vérifiables sont signalés comme angles morts (§1) et n'ont pas été inférés.